



中山大学计算机学院

人工智能

本科生实验报告

(2022 学年春季学期)

课程名称: Artificial Intelligence

教学班级	20240574	专业 (方向)	计算机科学与技术
学号	23320093	姓名	林宏宇

(说明: 本实验代码适合实验三和实验四的归结推理的所有题目, 因此一起提交)

一、实验题目

命题逻辑的归结推理

1. 命题逻辑的归结推理

编写函数 `ResolutionProp` 实现命题逻辑的归结推理。该函数要点如下:

- 输入为子句集(数据类型与格式详见课件), 每个子句中的元素是原子命题或其否定。
- 输出归结推理的过程, 每个归结步骤保存为字符串, 将所有归结步骤按序存入一个列表中并返回, 即返回的数据类型为 `list[str]`。
- 一个归结步骤的格式为 `步骤编号 R[用到的子句编号] = 子句`。如果一个子句包含多个公式, 则每个公式用编号 `a,b,c...` 区分, 如果一个子句仅包含一个公式, 则不用编号区分。(见课件和例题)

例子: 输入子句集

```
1 KB = {(FirstGrade,), (~FirstGrade,Child), (~Child,)}
```

则调用 `ResolutionProp(KB)` 后返回推理过程的列表如下:

```
1 1 (FirstGrade,),
2 2 (~FirstGrade,Child)
3 3 (~Child,),
4 4 R[1,2a] = (Child,),
5 5 R[3,4] = ()
```

二、实验内容

1. 算法原理

实现归结推理是自动推理的人工只能算法, 基于归结规则进行逻辑推导。其核心思想是将待证明命题转换为合取范式, 然后通过归结步骤不断推导新子句, 直至推导出空子句 ($\emptyset$), 证明原命题的结果。

实现的核心步骤:

- 标准化: 将知识库转换为一阶逻辑的子句形式。
- 合一: 实现最小合一, 用于匹配变量并替换。
- 归结: 找到可归结的两个子句, 通过合一消除相同谓词的互补文字, 生成新子句。
- 迭代归结: 重复进行归结步骤, 直到推导出空子句 ($\emptyset$) 或无法继续推理。



归结推理基本步骤:

(1) 标准化。将给定的知识库 KB 转换为子句范式, 子句是由若干文字组成的析取式 (如 $P(x) \vee Q(y) \vee \neg R(z)$) 表示为 $(P(x), Q(y), \neg R(z))$, 在实现时.

遵循以下表达:

- 使用集合 Set 存储子句集
- 每个子句用元组 Tuple 表示
- 文字和归结步用字符 Str 表示
- 非符号用波浪~表示

例如: $KB = \{('P(x)', 'Q(y)'), ('\neg Q(y)', 'R(z)'), ('\neg P(a)'),\}$ 表示子句集合 $\{P(x) \vee Q(y), \neg Q(y) \vee R(z), \neg P(a)\}$

(2) 归结操作, 归结操作是推理的核心步骤。

其基本规则是:

- 如果两个子句中存在互补文字, 即一个子句包含 $P(x)$, 另一个子句包含 $\neg P(x)$, 那么这两个子句可以进行归结, 生成一个新子句。
- 归结后的新子句由原子句中剩余的文字组成。
- 如果归结过程中产生了空子句 ($\emptyset$), 说明推理成功, 命题得证。

例如:

已知知识库中包含以下两个子句:

C1: $P(a) \vee Q(b)$ C2: $\neg P(a) \vee R(c)$

C1 和 C2 归结

得到: $Q(b) \vee R(c)$

如果最终得到空子句 $\emptyset$, 说明知识库中的命题是不可满足的, 从而证明原命题为真。

(3) 合一算法。在归结过程中, 如果遇到变量, 我们需要使用合一算法来确定如何替换变量, 使得两个文字可以匹配, 合一算法确保了归结推理可以在包含变量的情况下正常进行。

合一算法的基本规则:

- 如果两个文字完全相同, 则它们可以直接匹配。
- 建立一个变量集合, 其他文字默认为常量, 如果判断其中一个量是变量, 则用该变量替换另一个文字。
- 如果变量已经有替换规则, 则进行递归替换。

例如: $P(x)$ 和 $P(a)$ 可以合一, 合一变量替换 $\{x = a\}$

(4) 归结推理的迭代。归结推理是一个迭代过程。

具体流程如下:

- 选择两个子句进行归结, 尝试合一它们的文字。
- 如果可以合一, 则生成新子句, 并将其加入到子句集中。
- 如果新子句为空子句 ($\emptyset$), 推理结束, 命题得证。
- 否则, 重复步骤 1-3, 直到不再能产生新的子句, 或得出结论。



2. 伪代码

函数 `unify(term1, term2, theta)`:

- 将 `term1` 和 `term2` 替换为 `theta` 中的变量值 (`deep_substitute`)
- 将 `term1` 和 `term2` 用逗号分隔为列表 `term1_list` 和 `term2_list`
- 如果 `term1_list` 和 `term2_list` 长度不一致, 则返回 `None`
- 遍历 `term1_list` 和 `term2_list` 中的元素对:
 - 如果元素相等, 跳过
 - 如果元素是变量, 尝试统一 (`unify_var`)
 - 如果无法统一, 则返回 `None`
- 返回替换规则 `theta`

函数 `unify_var(var, term, theta)`:

- 如果 `var` 在 `theta` 中, 递归调用 `unify`
- 如果 `term` 在 `theta` 中, 递归调用 `unify`
- 如果 `var` 在 `term` 中, 返回 `None` (防止变量出现在自己的替换中)
- 将 `var` 和 `term` 绑定, 返回更新后的 `theta`

函数 `deep_substitute(term, theta)`:

- 递归替换 `term` 中的所有变量, 直到没有变量为止

函数 `occurs_check(var, term)`:

- 如果 `var == term`, 返回 `True`
- 如果 `term` 是逗号分隔的多个元素, 递归检查每个元素是否包含 `var`
- 返回 `False`

函数 `parse_predicate(expr)`:

- 解析谓词表达式, 将其转化为谓词名和参数列表

函数 `substitute(clause, theta)`:

- 根据 `theta` 替换子句中的变量, 返回替换后的子句

函数 `resolve(ci, cj, i, j)`:

- 遍历 `ci` 和 `cj` 中的所有文字配对:
 - 如果有互补文字, 则尝试统一它们的参数
 - 如果统一成功, 生成新的子句并返回归结步骤标号和新子句
- 返回 `None` 如果无法归结

函数 `format_substitution(theta)`:

- 将替换规则 `theta` 格式化为 '`{var1=value1, var2=value2, ...}`' 的形式

函数 `resolution(KB)`:

- 将知识库 `KB` 转化为列表形式的子句



```
        if new_clause_tuple not in seen_clauses:
            steps.append(f"{step_num} {step_label} {new_clause}")
            new_clauses.append(new_clause_tuple)
            seen_clauses.add(new_clause_tuple)
            step_num += 1

    if not new_clauses:
        break

    clauses.extend(new_clauses)  #将初步归结得到的句子放回子句集

return steps
```

(2) 归结核心部分 归结两个句子

```
"""对两个子句进行归结"""
def resolve(ci: Tuple[str, ...], cj: Tuple[str, ...], i: int, j: int) -> Optional[Tuple[str, List[str]]]:
    for ai, lit1 in enumerate(ci):
        for aj, lit2 in enumerate(cj):
            #拆解句子，得到谓词和参数
            if lit1.startswith("~"):
                pred1, args1 = parse_predicate(lit1[1:])
                pred2, args2 = parse_predicate(lit2)
                negated = True
            else:
                pred1, args1 = parse_predicate(lit1)
                pred2, args2 = parse_predicate(lit2[1:])
                negated = lit2.startswith("~")

            if negated and pred1 == pred2: #判断是否可以归结
                theta = unify(".".join(args1), " ".join(args2), {})
                if theta is not None:
                    new_clause = substitute(tuple(set(ci + cj) - {lit1, lit2}), theta)
                    formatted_theta = format_substitution(theta) #归结步骤标签
                    #添加[]内标号格式
                    if len(ci) == 1 and len(cj) == 1:
                        step_label = f"R[{i+1},{j+1}] {formatted_theta} = "
                    elif len(ci) == 1: #只有 ci 一个谓词 使用标号 j+1 和子标号 aj
                        step_label = f"R[{i+1},{j+1}{chr(97+aj)}] {formatted_theta} = "
                    elif len(cj) == 1:
                        step_label = f"R[{i+1}{chr(97+ai)},{j+1}] {formatted_theta} = "
                    else: #正常情况
                        step_label = f"R[{i+1}{chr(97+ai)},{j+1}{chr(97+aj)}] {formatted_theta} = "
                    return step_label.strip(), new_clause

    return None
```



(3) 实现最小合一算法（MGU），返回替换规则

```
"""实现最小合一算法，返回替换规则"""  
  
def unify(term1: str, term2: str, theta: Dict[str, str]) -> Optional[Dict[str, str]]:  
    term1, term2 = deep_substitute(term1, theta), deep_substitute(term2, theta)  
    term1_list, term2_list = term1.split(","), term2.split(",")  
    #将 term1 和 term2 按逗号分隔，防止 'tony,snow' 这样的字符串无法解析  
  
    if len(term1_list) != len(term2_list):  
        return None  
    #逐个统一参数 theta 统一参数记录替换规则  
    for t1, t2 in zip(term1_list, term2_list):  
        if t1 == t2: #参数相同 跳过  
            continue  
        elif t1 in VAR_SET:  
            theta = unify_var(t1, t2, theta)  
        elif t2 in VAR_SET:  
            theta = unify_var(t2, t1, theta)  
        else:  
            return None #无法统一  
    if theta is None:  
        return None  
  
    return theta
```

4. 创新点&优化（如果有）

(1) 输出时去冗余优化，将那些对于得到最终结果空集无用的句子删除，不展示在最终结果中，使得推理过程简洁明了。通过 `remove_redundant_steps()` 方法追溯关键推理路径，仅保留必要的归结步骤，减少无效计算。

过滤前的归结推理过程：

```
1 ('GradStudent(sue)',)  
2 ('~HardWorker(sue)',)  
3 ('~Student(x)', 'HardWorker(x)')  
4 ('~GradStudent(x)', 'Student(x)')  
5 R[1,4a] {'x':'sue'} = ('Student(sue)',)  
6 R[2,3b] {'x':'sue'} = ('~Student(sue)',)  
7 R[3a,4b] = ('HardWorker(x)', '~GradStudent(x)')  
8 R[1,7b] {'x':'sue'} = ('HardWorker(sue)',)  
9 R[2,7a] {'x':'sue'} = ('~GradStudent(sue)',)  
10 R[5,6] = ∅
```

过滤后的归结推理过程：

```
1 ('GradStudent(sue)',)
2 ('~HardWorker(sue)',)
3 ('~Student(x)', 'HardWorker(x)')
4 ('~GradStudent(x)', 'Student(x)')
5 R[1,4a] {'x'='sue'} = ('Student(sue)',)
6 R[2,3b] {'x'='sue'} = ('~Student(sue)',)
7 R[5,6] = ∅
```

(2) 参数解析：

原来在判断参数是否相同经常出错，因为前面的例子谓词只有一个参数，则做调整，将（）内的字符串按逗号分割得到多个参数进行后续判断。

```
term1, term2 = deep_substitute(term1, theta), deep_substitute(term2, theta)
term1_list, term2_list = term1.split(","), term2.split(",")
#将 term1 和 term2 按逗号分隔，防止 'tony,snow' 这样的字符串无法解析
```

(3) 去重方法：

先按照遍历归结出所有得到的句子，再从结果空集出发，进行去重操作，删除对得到结果没有帮助的句子，输出归结推理过程。

三、实验结果及分析

1. 实验结果展示示例（可图可表可文字，尽量可视化）

示例 1

输入知识库：

```
KB={('FirstGrade',),('~FirstGrade','Child'),('~Child',)}
```

归结推理输出：

```
1 ('~FirstGrade', 'Child')
2 ('~Child',)
3 ('FirstGrade',)
4 R[1a,3] = ('Child',)
5 R[2,4] = ∅
```

可以看出，成功推出空子句 $\emptyset$ ，证明知识库的命题为真。

示例 2

输入知识库：

```
KB = {('~HardWorker(sue)',), ('GradStudent(sue)',),
      ('~GradStudent(x)', 'Student(x)'),
```



```
('~Student(x)', 'HardWorker(x)')}
```

归结推理输出：

```
1 ('GradStudent(sue)',)
2 ('~HardWorker(sue)',)
3 ('~GradStudent(x)', 'Student(x)')
4 ('~Student(x)', 'HardWorker(x)')
5 R[1,3a] {'x'='sue'} = ('Student(sue)',)
6 R[2,4b] {'x'='sue'} = ('~Student(sue)',)
7 R[5,6] = ∅
```

可以看出，成功推出空子句 $\emptyset$ ，证明知识库的命题为真。

示例 3

输入知识库：

```
KB = {
  ('A(tony)',), ('A(mike)',), ('A(john)',),
  ('L(tony,rain)',), ('L(tony,snow)',),
  ('~A(x)', 'S(x)', 'C(x)'),
  ('~C(y)', 'L(y,rain)'),
  ('L(z,snow)', 'S(z)'),
  ('~L(tony,u)', 'L(mike,u)'),
  ('L(tony,v)', 'L(mike,v)'),
  ('~A(w)', 'C(w)', 'S(w)')
}
```

归结推理输出：

```
归结推理过程：
1 ('L(tony,snow)',)
2 ('L(tony,v)', 'L(mike,v)')
3 ('~L(tony,u)', 'L(mike,u)')
4 R[2a,3a] {'v'='u'} = ('L(mike,u)', 'L(mike,u)')
5 R[3b,4a] = ('~L(tony,u)',)
6 R[1,5] {'u'='snow'} = ∅
```

可以看出，成功推出空子句 $\emptyset$ ，证明知识库的命题为真。

示例 4

输入知识库：

```
KB={('On(tony,mike)',),('On(mike,john)',),
  ('Green(tony)',),('~Green(john)',),
```




```
('~On(xx,yy)', '~Green(xx)', 'Green(yy)')}]
```

归结推理输出：

```
1 ('On(tony,mike)',)
2 ('On(mike,john)',)
3 ('Green(tony)',)
4 ('~On(xx,yy)', '~Green(xx)', 'Green(yy)')
5 ('~Green(john)',)
6 R[3,4b] {'xx'='tony'} = ('Green(yy)', '~On(tony,yy)')
7 R[4c,5] {'yy'='john'} = ('~Green(xx)', '~On(xx,john)')
8 R[1,6b] {'yy'='mike'} = ('Green(mike)',)
9 R[2,7b] {'xx'='mike'} = ('~Green(mike)',)
10 R[8,9] = ∅
```

可以看出，成功推出空子句 $\emptyset$ ，证明知识库的命题为真。

2. 评测指标展示及分析（机器学习实验必须有此项，其它可分析运行时间等）

（1）案例一

```
Resolution 平均运行时间: 0.055030 秒
remove_redundant_steps 平均运行时间: 0.000808 秒

归结推理过程:
1 ('A(mike)',)
2 ('~A(x)', 'S(x)', 'C(x)')
3 ('L(tony,snow)',)
4 ('~L(tony,u)', '~L(mike,u)')
5 ('L(z,snow)', '~S(z)')
6 ('~A(w)', '~C(w)', 'S(w)')
7 R[2c,6b] {'x'='w'} = ('S(w)', '~A(w)')
8 R[3,4a] {'u'='snow'} = ('~L(mike,snow)',)
9 R[1,7b] {'w'='mike'} = ('S(mike)',)
10 R[5a,8] {'z'='mike'} = ('~S(mike)',)
11 R[9,10] = ∅
```

运行时间分析：

Resolution 平均运行时间: 0.055030 秒

remove_redundant_steps 平均运行时间: 0.000808 秒

归结推理（Resolution）耗时较长，主要因为案例一的子句集较大，涉及多个归结步骤（共 11 步）。去冗余步骤（remove_redundant_steps）耗时极短，仅占 Resolution 时间的 1.47%，说明该步骤的算法效率较高，对整体计算时间影响不大。

归结推理步骤分析：

归结推理 共 11 步，最终得到空子句（ $\emptyset$ ），说明逻辑证明成功。

由于推理链较长，Resolution 需要进行多次变量替换、子句合并，导致计算量增加。

关键归结步骤：

第 7 步 由子句 2 和 6 归结出 $S(w), \sim A(w)$ ，说明 $S(w)$ 成立。

第 9 步 进一步证明 $S(\text{mike})$ 成立。

第 11 步 归结出空子句，说明假设可证。

(2) 案例二

```
Resolution 平均运行时间: 0.000073 秒
remove_redundant_steps 平均运行时间: 0.000050 秒

归结推理过程:
1 ('~Child',)
2 ('FirstGrade',)
3 ('~FirstGrade', 'Child')
4 R[2,3a] = ('Child',)
5 R[1,4] =  $\emptyset$ 
```

运行时间分析：

Resolution 平均运行时间：0.000073 秒

remove_redundant_steps 平均运行时间：0.000050 秒

归结推理过程非常短（仅 5 步），导致计算时间极短，主要原因是 初始子句较少，推理链路较短。

由于推理过程简单，去冗余步骤所需的时间接近 Resolution 本身，优化空间有限。

归结推理步骤分析：

归结推理 仅 5 步，快速推导出空子句 ($\emptyset$)。

关键归结步骤：

第 4 步 由子句 2 和 3 归结出 Child

第 5 步 进一步归结 $\sim\text{Child}$ ，推导出矛盾，证明完成。

由于案例二的逻辑链路较短，Resolution 过程非常快，仅进行两次归结。

(3) 总结

案例一推理链更长，Resolution 运行时间显著增加（0.055030 s）。

案例二归结步骤少，计算时间极短（0.000073 s）。

去冗余操作耗时很小，对整体性能影响不大。

优化重点应放在 Resolution 算法，包括归结策略、unify 函数和子句管理方式。

四、 思考题

1.归结推理的优化与冗余消除

在实验中，我们实现了基于合一算法和子句归结的推理过程。然而，归结推理会生成大量冗余子句，影响计算效率。因此，我们在 remove_redundant_steps 函数中进行了冗余步骤的消除，仅保留必要的推导路径。实验表明，这种优化能



显著减少归结推理的计算复杂度，提高执行效率。

2.合一算法的正确性对推理的影响

`unify` 函数实现了最小合一算法，用于匹配不同子句中的变量。实验中发现，若合一过程存在错误（如未正确替换变量或变量-项不匹配），则归结可能无法正确进行，甚至导致错误推导。因此，确保合一算法的健壮性是归结推理能否正确运行的关键。

3.归结推理在知识库中的应用

本次实验使用了多个知识库示例，涵盖逻辑推理、事实推导等场景。
`KB={('On(tony,mike)'),('On(mike,john)'),('Green(tony)'),('~Green(john)'),('~On(xx,yy)'),('~Green(xx)'),('Green(yy)')}`中，系统成功推导出了 `Green(mike)` 的结论。这说明归结推理可以在自动定理证明、专家系统、人工智能推理等场景中发挥作用，具有重要的实际意义。

五、 参考资料

1. 上课课件

2. https://blog.csdn.net/qq_44159028/article/details/120575621
3. <https://blog.csdn.net/aa122509969/article/details/125421813>
4. https://blog.csdn.net/m0_70885101/article/details/125999020

一、 实验题目

最一般合一算法

2.最一般合一算法

编写函数 `MGU` 实现最一般合一算法。该函数要点如下：

- 输入为两个原子公式，它们的谓词相同。其数据类型为 `str`，格式详见课件。
- 输出最一般合一的结果，数据类型为 `dict`，格式形如{变量：项，变量：项}，其中的变量和项均为字符串。
- 若不存在合一，则返回空字典。

例子：

调用 `MGU('P(xx,a)', 'P(b,yy)')` 后返回字典 `{'xx':'b', 'yy':'a'}`。

调用 `MGU('P(a,xx,f(g(yy)))', 'P(zz,f(zz),f(uu))')` 后返回字典 `{'zz':'a', 'xx':'f(a)', 'uu':'g(yy)'}`。

二、实验内容

1. 算法原理

(1) 合一是一种用于逻辑推理和模式匹配的基本技术，它的目标是找到一个替换，使得两个表达式变得相等。合一常用于一阶逻辑推理、逻辑编程、自动定理证明等领域。在合一过程中，变量可以绑定到某个具体值或另一个变量，而非变量如常量必须完全匹配。

例如：

- $P(xx, a)$ 和 $P(b, yy)$ 可以合一，得到替换 $\{xx: 'b', yy: 'a'\}$ 。
- $P(a, xx, f(g(yy)))$ 和 $P(zz, f(zz), f(uu))$ 需要通过多步变量替换进行合一。

2. 代码解析

最小合一算法核心步骤包括：

(1) 变量、谓词、参数解析

- 判断一个符号是否为变量。
- 判断一个表达式是否为复合谓词。
- 解析复合谓词，获取其名称和参数列表。
- 按逗号正确分割谓词的参数。

(2) 递归替换机制

- 变量绑定后，可能影响整个表达式，因此需要递归替换。
- 变量绑定时，若已经存在替换关系，需要递归解析。

(3) 合一逻辑

- 若两个项相等，返回当前替换 θ 。
- 若 x 是变量，则将其替换。
- 若 y 是变量，则交换并替换。
- 若 x 和 y 均为复合项，则解析其谓词和参数，逐个进行递归合一。
- 若 x 和 y 既不是变量也不是复合项，则匹配失败。

3. 伪代码

Function RESOLUTION(KB)

CLAUSES $\leftarrow$ KB, STEPS $\leftarrow$ [], SEEN_CLAUSES $\leftarrow$ KB

step_num $\leftarrow$ |CLAUSES| + 1

While True do

NEW_CLAUSES $\leftarrow$ []

For each (C1, C2) in CLAUSES do

result $\leftarrow$ RESOLVE(C1, C2)

If result is not None then

(step_label, new_clause) $\leftarrow$ result

If new_clause is empty then

Add "step_num step_label $\varnothing$ " to STEPS

Return STEPS



```
If new_clause not in SEEN_CLAUSES then
    Add "step_num step_label new_clause" to STEPS
    Add new_clause to NEW_CLAUSES, SEEN_CLAUSES
    step_num  $\leftarrow$  step_num + 1
If NEW_CLAUSES is empty then Return "Fail"
Add NEW_CLAUSES to CLAUSES
```

Function RESOLVE(C1, C2)

```
For each L1 in C1, L2 in C2 do
    If L1 and L2 are complementary then
        theta  $\leftarrow$  UNIFY(L1, L2)
        If theta is not None then
            C'  $\leftarrow$  (C1  $\cup$  C2) - {L1, L2}, Apply theta to C'
            Return ("R[C1_id, C2_id]", C')
Return None
```

Function UNIFY(term1, term2)

```
If term1 = term2 then Return {}
If term1 or term2 is a variable then Return {term1: term2} if valid
If term1, term2 are functions with same name, args then
    Return merge UNIFY results for each subterm pair
Return None
```

4. 关键代码展示（带注释）

合一算法

```
"""合一算法"""
def unify(x, y, theta):
    if theta is None:
        return None

    #应用替换规则 theta 到 x 和 y
    x = apply_substitution(x, theta)
    y = apply_substitution(y, theta)

    if x == y:
        return theta

    elif is_variable(x):#x 是变量
        return unify_var(x, y, theta)

    elif is_variable(y):#y 是变量
        return unify_var(y, x, theta)

    elif is_compound(x) and is_compound(y):#x 和 y 都是复合项
        #解析谓词，返回谓词名和参数列表
        x_pred, x_args = parse_predicate(x)
        y_pred, y_args = parse_predicate(y)
```

```
if x_pred != y_pred or len(x_args) != len(y_args):#谓词名不同或参数个数不同
    return None

for x_i, y_i in zip(x_args, y_args):#逐个统一参数
    theta = unify(x_i, y_i, theta)

    if theta is None:
        return None

return theta

else:
    return None
```

5. 创新点&优化（如果有）

（1）高效变量替换机制

在 `unify_var()` 和 `apply_substitution()` 函数中,使用 递归替换 和 变量绑定更新,确保所有变量的替换是最新的,减少了不必要的重复计算。

（2）避免无限递归

通过 `occurs_check()` 防止变量在自己的替换表达式中出现,避免了无限递归（例如 `xx=f(xx)` 的情况）。

（3）提升 `split_args()` 解析能力

采用 嵌套深度计数,能够正确解析包含括号嵌套的参数（如 `f(g(yy))`），确保合一算法可以处理更复杂的谓词。

（4）统一变量管理

采用 `apply_substitution()` 在 每次变量绑定后,更新整个替换规则集,减少后续合一过程中的查询开销。使得 `unify_var()` 能够及时传播变量绑定,减少 `unify()` 过程中不必要的计算。

（5）减少重复计算

`apply_substitution()` 在每次查询时先检查 `theta`, 如果变量已经有绑定,就直接替换,而不是重复调用 `unify()`。

三、 实验结果及分析

1. 实验结果展示示例（可图可表可文字, 尽量可视化）

案例一

输入:

```
KB = {'P(a,xx,f(g(yy)))', 'P(zz,f(zz),f(uu))'}
```

输出:

```
{'zz': 'a', 'xx': 'f(a)', 'uu': 'g(yy)'}
```

案例二

输入:

```
KB = {'P(xx,a)', 'P(b,yy)'}
```

输出:

```
{'xx': 'b', 'yy': 'a'}
```

2. 评测指标展示及分析（机器学习实验必须有此项，其它可分析运行时间等）

(1) 案例分析

```
{'xx': 'b', 'yy': 'a'}
```

Unify 平均运行时间: 0.000036 秒

该案例相对简单，仅涉及基础变量的合一操作，且不包含复合项，因此运行时间显著低于第二个案例。

```
{'zz': 'a', 'xx': 'f(a)', 'uu': 'g(yy)'}
```

Unify 平均运行时间: 0.000138 秒

该案例中的合一操作涉及了多个变量和复杂的复合项，如 $f(a)$ 和 $g(yy)$ 。虽然包含了复合项，但每个变量的映射关系相对简单，因此时间略高，但仍在可接受范围内。

(2) 时间对比与优化思考

在这两个案例中，合一算法的运行时间都较短，但对于包含复杂复合项（如 $f(a)$ 和 $g(yy)$ ）的情况，运行时间略有增加。这个现象表明，复杂的表达式需要更多的计算步骤来处理，因此运行时间随输入的复杂性增加。

对于简单的变量替换，合一过程非常高效，时间开销较低。

四、 思考题

1. 关于算法效率的思考

当前合一算法在变量替换时采用递归方式，尽管能够保证正确性，但在处理深层嵌套项时，可能导致较高的时间复杂度。如果知识库规模较大或推理深度较深，算法的性能可能会成为瓶颈。未来可以考虑使用更高效的数据结构（如并查集）来优化变量绑定管理，减少递归调用的次数，从而提升合一过程的效率。

2. 关于归结推理的可扩展性

本实验实现了基础的一阶逻辑归结推理，但实际应用中，推理过程往往涉及更复杂的规则集合，如具有函数符号的复杂项或量词推理。在更大规模的知识库或现实应用中，需要结合启发式搜索、索引优化等技术，以减少不必要的归结步骤，提高推理的可扩展性和实际可用性。

五、 参考资料

此部分是从上一个实验的合一替换部分单独分出的一部分，原理相同，参考资料相同。